MIT 8103 Advanced Database Systems

IsaacMart Database Portfolio Summary

Name: Idowu Isaac
Student ID: 30168852
Cohort: September 2026
Repository: https://github.com/LaProfessor/Idowu_Isaac_30168852_MIT8103

Case study and scope

IsaacMart is a simulated Nigerian e-commerce retailer. Customers place orders, warehouse staff manage products and stock, finance staff reconcile payments, and managers analyse sales. PostgreSQL stores customers, categories, products, orders, order items and payments in Nigerian naira. MongoDB stores flexible product catalogue attributes. All data is synthetic. Delivery, taxes, refunds and customer authentication are outside this demonstration.

Portfolio 1: Database Design and Modelling

The relational model contains six tables: Customers, Categories, Products, Orders, OrderItems and Payments. Primary keys identify records, while foreign keys connect related records. Unique constraints protect customer emails, product SKUs and payment references. CHECK constraints reject negative stock, invalid quantities, invalid prices and unsupported statuses.

OrderItems resolves the many-to-many relationship between orders and products. Its composite primary key is (order_id, product_id). The design separates customer, product and category details to remove partial and transitive dependencies, supporting third normal form under the stated business rules.

Historical unit_price is stored in OrderItems because it represents the price charged when the order was placed. Changing a product's current price must not alter previous orders. Order totals are calculated from quantity and historical unit price. Cross-row rules, such as matching payment totals to order values, require application checks and reconciliation.

I created the schema and loaded the supplied synthetic dataset in GitHub Codespaces. My count checks confirmed 10,000 customers, 1,000 products, 200,000 orders, 200,000 order items and 180,000 payments. Integrity tests rejected negative stock, missing customer references and duplicate customer emails. The design folder contains the schema, seed script, ER diagram and normalisation explanation.

Portfolio 2: Query Processing and Optimisation

The customer-history query filters orders by customer and returns the latest ten. Before indexing, PostgreSQL used a parallel sequential scan followed by sorting. After creating a composite B-tree index on (customer_id, ordered_at DESC) and refreshing statistics, PostgreSQL used an index scan.

In my Codespace test, execution time changed from **13.326 ms to 0.131 ms**, approximately a 102-fold improvement in one comparison. Top-level execution buffers changed from 1,311 shared hits to 10 shared hits and 3 shared reads. The index helped locate matching orders in the required order and stop after ten rows.

These are single-run student measurements, not averages or production performance predictions. Separate initial assistant-run measurements recorded medians of 12.802 ms before indexing and 0.093 ms afterward across five measured runs following a warm-up. The two sets of results are labelled separately. Cache state and system load affect timings, and indexes require storage and write maintenance.

I also executed queries for category sales, monthly sales, customers without orders and payment reconciliation. The reconciliation query returned zero mismatches. Student-prefixed files contain my query results and before/after execution plans.

Portfolio 3: Transactions and Concurrency

I verified that COMMIT preserves changes and ROLLBACK discards uncommitted changes. A committed two-unit decrement changed product 1 stock from 100 to 98. A subsequent five-unit decrement temporarily changed it to 93; rollback restored 98.

The initial assistant-run locking test used a 500 ms timeout. In my own two-session Codespace test, the first session held a product-row lock while the second session reached its two-second lock timeout. Rolling back the first transaction restored stock to 98. This demonstrated contention between concurrent updates; the timeout limited waiting rather than demonstrating deadlock prevention.

In my REPEATABLE READ test, the first transaction read product 2 stock as 100. Another session committed an increase to 101, but the first transaction continued to read 100 from its existing snapshot. After it committed, its next query returned 101. I then restored the demonstration stock to 100.

The corrected checkout script groups stock reservation, order creation, order items and a pending payment in one transaction. It aborts when the guarded stock update affects no row and derives the payment amount from order lines. My successful-path test showed matching order and payment values of ₦1,050. Rollback restored stock to 98 and left zero demonstration orders. The insufficient-stock branch has not yet been separately verified.

An external payment gateway cannot participate directly in this local transaction. A production design would require idempotent payment handling, an outbox or saga workflow, and reconciliation.

Portfolio 4: NoSQL and Advanced Data Models

MongoDB stores product documents with embedded attributes. Headphones have battery hours, clothing has size and material, and lamps have wattage. This supports varying product descriptions without requiring many nullable relational columns.

I executed the catalogue script and confirmed three inserted documents, a successful battery-hours query and an attribute update. The tests rejected a duplicate SKU with error code 11000 and invalid attributes with validation error code 121.

PostgreSQL remains authoritative for prices, stock, orders and payments. MongoDB stores descriptive catalogue attributes. Cross-store synchronisation is discussed but not implemented, and no cross-store foreign-key or atomic transaction guarantee is claimed.

Portfolio 5: Distributed and Cloud Database Exercise

This activity follows the assessment's permitted discussion option. The proposed design uses a managed PostgreSQL primary, a cross-zone standby and asynchronous reporting replicas, alongside a MongoDB replica set for catalogue data.

Synchronous replication can reduce acknowledged-transaction loss but adds latency and may reduce write availability when required acknowledgements cannot be obtained. Asynchronous replicas can serve reporting workloads but may return stale data. Checkout should use authoritative stock information. Failover requires fencing to prevent conflicting writers, while backups and restoration drills remain necessary because replication does not replace backups.

Customer-based sharding could co-locate customer orders but would complicate shared inventory and cross-shard checkout. Date partitioning can support retention without necessarily distributing data across servers. The demonstration dataset does not establish a need for sharding. No cloud deployment, replication experiment, sharding implementation or failover measurement was performed.

Evidence and Repository

The private repository contains scripts, explanations, an ER diagram, execution plans and outputs. Initial assistant-run evidence is retained separately from Student-prefixed Codespace evidence. My initial student verification files were committed and pushed in commit bf95174; corrected checkout evidence was added afterward.

The complete first-session isolation sequence was shared during guided verification. Its saved final-state file records the final value only. Assessor access must be confirmed before submission. GitHub activity records actual work dates; no semester-long development history is claimed.

Individual Technical Reflection

I understood why historical unit prices must remain separate from current product prices. I also learned that a suitable index can reduce the work required for a selective query, although one timing comparison does not establish performance for every workload.

The transaction exercises helped me distinguish committed changes from temporary changes. Keeping one transaction open while another session attempted an update made row locking observable. The repeatable-read exercise showed why a transaction can continue seeing an earlier value after another session commits.

Saving evidence from the container was challenging. A relative output path failed inside `psql` because the repository folders were outside the database container. With guidance, I saved output from the Codespace shell using `tee`. I also had to coordinate two terminals correctly. I would improve the exercise by repeating performance measurements, testing more query filters and separately testing insufficient-stock checkout rejection.

AI Use Declaration

I adopted the simulated IsaacMart case study suggested by OpenAI ChatGPT/Codex. AI assisted with the initial schemas, scripts, synthetic data, diagrams, explanations, analysis and documentation, and performed initial automated checks in its own environment.

I subsequently ran the setup, SQL queries, index comparison, commit/rollback, locking, repeatable-read isolation, MongoDB, integrity and corrected checkout exercises in my GitHub Codespace with step-by-step AI guidance. I shared outputs for review and saved student-run evidence. AI also helped revise my reflection and correct technical wording. I remain responsible for reviewing, understanding and explaining the submitted work.

References

1. Miva University. *MIT 8103 Advanced Database Systems: CA September Cohort 2026*. Supplied assessment brief.
2. PostgreSQL Global Development Group. *PostgreSQL 17 Documentation*: Constraints, Using EXPLAIN, Multicolumn Indexes, Transaction Isolation, Explicit Locking and Warm Standby. https://www.postgresql.org/docs/17/
3. MongoDB. *MongoDB Manual*: Data Modeling, Schema Validation and Replication. https://www.mongodb.com/docs/manual/